# Function in Python

## Function:

In Python, the function is a block of code defined with a name. We use functions whenever we need to perform the same task multiple times without writing the same code again. Using def keyword with function name.It can take arguments and returns the value.

## Function Code Example

Here's a simple function example:

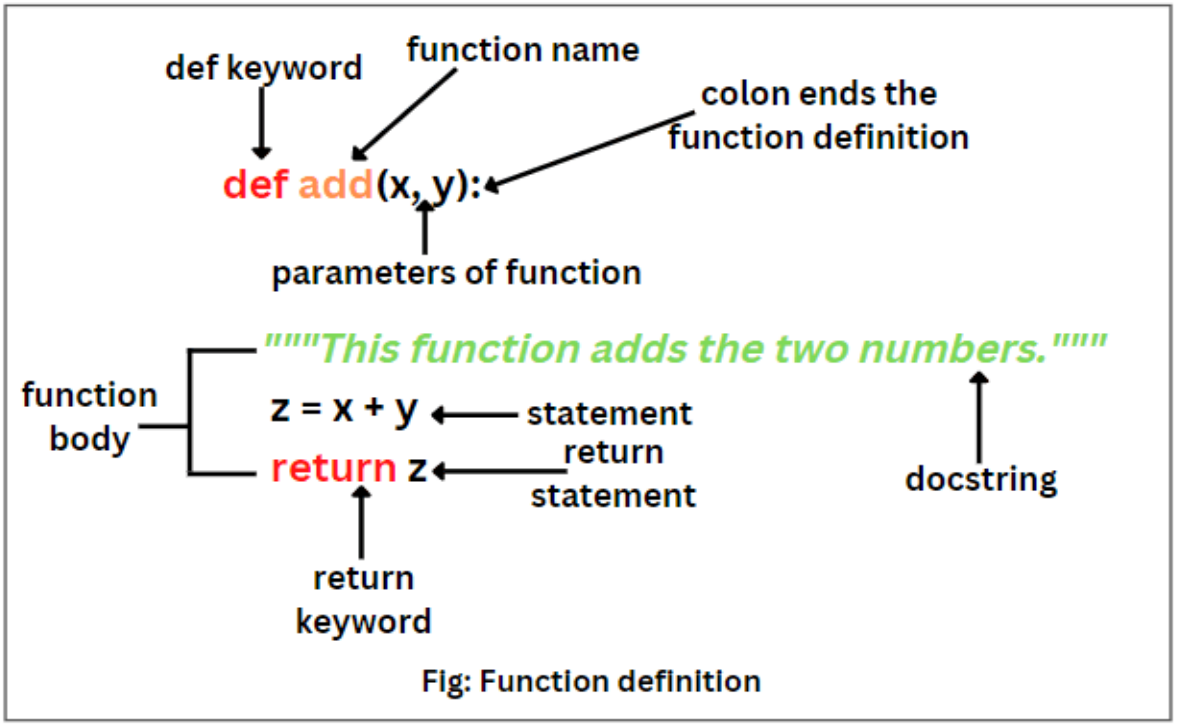
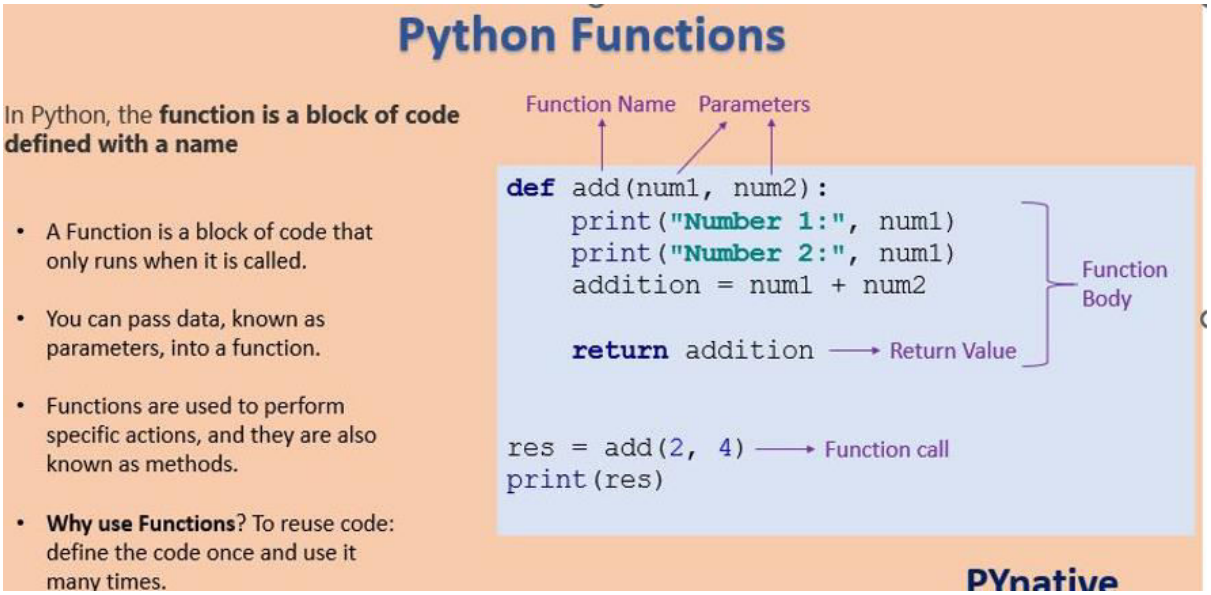


Fig: Function definition

In Python, the function is a block of code defined with a name. We use functions whenever we need to perform the same task multiple times without writing the same code again. It can take arguments and returns the value.

Function improves efficiency and reduces errors because of the reusability of a code. Once we create a function, we can call it anywhere and anytime. The benefit of using a function is reusability and modularity.

```
# Function to add two numbers
def add(a, b):
    return a + b
result = add(5, 3)   # Calling the function
print(result)        # Output will be 8
```




## Where Are Functions Applicable?

Functions are applicable in many situations, such as:

- **Reusability**: When you want to reuse a block of code multiple times.

- **Modularity**: To break down large problems into smaller, manageable parts.
- **Readability**: Functions improve code readability by organizing code logically.
- **Testability**: It makes unit testing easier by testing individual functions.

**Types of Functions in Python**

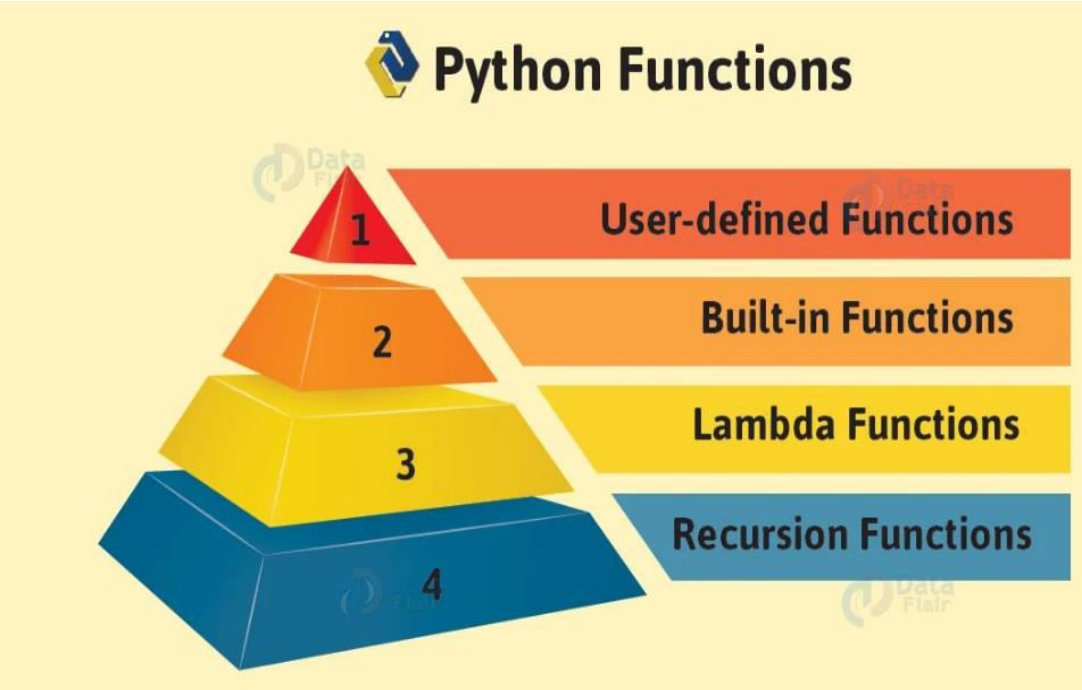
Functions in Python are reusable blocks of code that perform a specific task. They help in making the code more modular, readable, and manageable.

There are mainly two types of functions in Python:

- **Built-in Functions**: Functions that are provided by Python (e.g., print(), len(), type()).
- **User-defined Functions**: Functions that you define yourself using the def keyword.

There are also different ways functions can be categorized:

- **Function with parameters**: Functions that take input.
- **Function without parameters**: Functions that don't take input.
- **Function with return value**: Functions that return a result.
- **Function without return value**: Functions that don’t return anything.




**Built-in Functions**

These are functions that come pre-defined in Python, and you can use them directly without defining them.

Some commonly used built-in functions include:

- print()
- len()
- type()
- sum()
- max()
- min()
- abs(), etc.

## 1. `print()` – Prints Output

This function displays text or values on the screen.

```python
print("Hello, Python!")
```

**Output:**

```
Hello, Python!
```

## 2. `len()` – Returns Length

This function returns the length (number of elements) in a sequence (like a string, list, or tuple).

```python
word = "Python"
print(len(word))  # Output: 6

numbers = [10, 20, 30, 40]
print(len(numbers))  # Output: 4
```

## 3. `type()` – Returns Type of a Variable

This function returns the data type of the given object.

```python
print(type(10))       # Output: <class 'int'>
print(type(3.14))     # Output: <class 'float'>
print(type("Hello"))  # Output: <class 'str'>
print(type([1, 2, 3]))  # Output: <class 'list'>
```

## 4. `sum()` – Returns Sum of Elements

This function returns the sum of all elements in a list, tuple, or other iterable.

```python
numbers = [1, 2, 3, 4, 5]
print(sum(numbers))  # Output: 15

print(sum((10, 20, 30)))  # Output: 60
```

## 5. `max()` – Returns Maximum Value

This function returns the largest value in a given list or tuple.

```python
numbers = [4, 8, 15, 16, 23, 42]
print(max(numbers))  # Output: 42

print(max("apple", "banana", "cherry"))  # Output: cherry (based on alphabetical order)
```

## 6. `min()` – Returns Minimum Value

This function returns the smallest value in a given list or tuple.

```python
numbers = [4, 8, 15, 16, 23, 42]
print(min(numbers))  # Output: 4

print(min("apple", "banana", "cherry"))  # Output: apple
```

### 7. `abs()` – Returns Absolute Value

This function returns the absolute (positive) value of a number.

```python
print(abs(-10))  # Output: 10
print(abs(5))    # Output: 5
print(abs(-3.14))  # Output: 3.14
```

### Other Useful Built-in Functions

- `round(x, n)` : Rounds a number `x` to `n` decimal places.
- `sorted(iterable)` : Returns a sorted list from the given iterable.
- `pow(a, b)` : Returns `a` raised to the power `b` ( `a^b` ).
- `input()` : Takes input from the user.
- `range(n)` : Generates a sequence of numbers.

## User-defined Functions

User-defined Functions with no parameter:

These are functions that you define yourself using the def keyword. You can define a function and use it multiple times.

### Example of a Simple User-defined Function:

```python
def greet():
    print("Hello, welcome to Python!")

greet()  # Calling the function
```

**Output:**

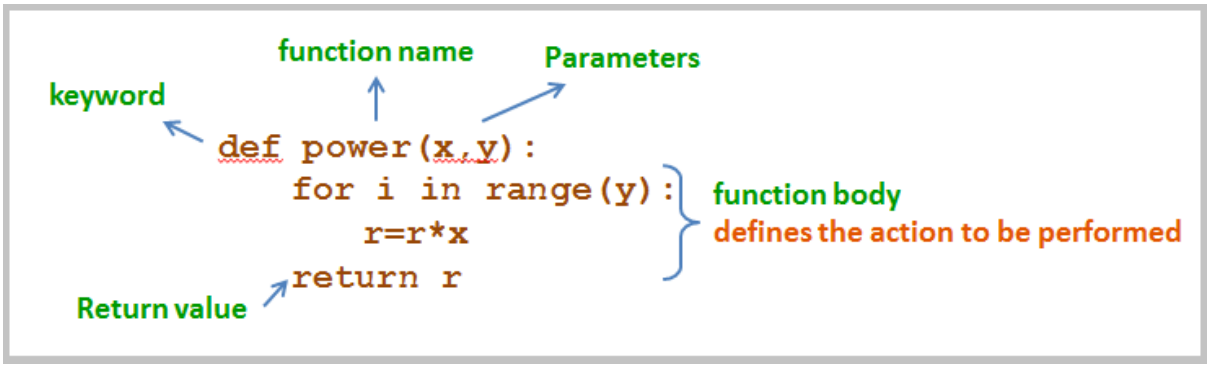
```css
Hello, welcome to Python!
```

## Categorization of Functions

User-defined functions can be categorized based on parameters and return values.

### A. Function with Parameters

A function that accepts input values (arguments) when called.



**Example:**

```python
def add_numbers(a, b):
    sum = a + b
    print("Sum:", sum)

add_numbers(5, 10)  # Passing arguments 5 and 10
```

**Output:**

```makefile
Sum: 15
```

Here, `a` and `b` are parameters, and `5` and `10` are arguments.

## C. Function with Return Value

A function that processes some data and returns a result.

**Example:**

```python
def multiply(a, b):
    return a * b  # Returns the product of a and b

result = multiply(4, 5)
print("Multiplication Result:", result)
```

**Output:**

```rust
Multiplication Result: 20
```

Here, the function returns the product, which can be used later in the program.

## D. Function without Return Value

A function that performs an operation but doesn't return anything.

**Example:**

```python
def greet_user(name):
    print("Hello,", name)

greet_user("Alice")
```

**Output:**

```
Hello, Alice
```

Here, the function just prints a message but doesn't return any value.

### Summary

| Type of Function | Description | Example |
|---|---|---|
| **Built-in Functions** | Predefined in Python | `print()`, `len()`, `type()` |
| **User-defined Functions** | Functions created by the user using `def` | Custom functions |
| **With Parameters** | Takes input values | `def add(a, b):` |
| **Without Parameters** | No input values | `def greet():` |
| **With Return Value** | Returns a result | `def multiply(a, b): return a * b` |
| **Without Return Value** | Performs an action but doesn't return | `def greet_user(name): print(name)` |

## How Functions Are Implemented in Python

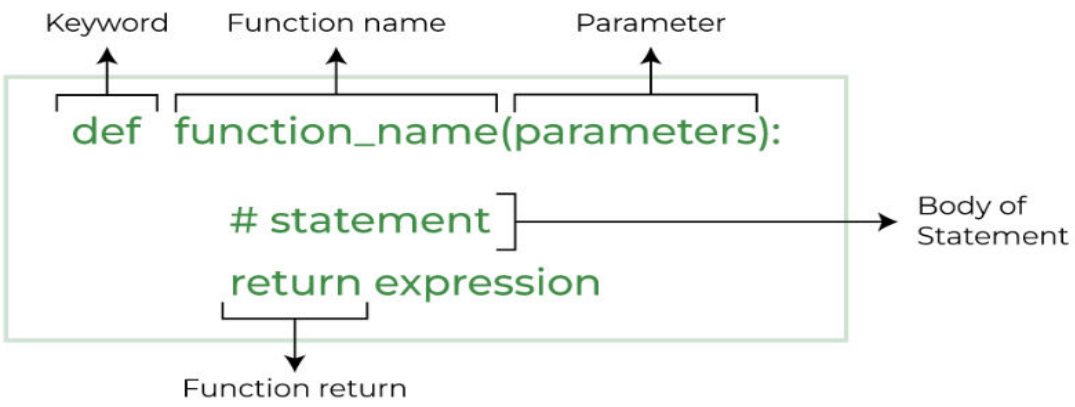
In Python, functions are implemented using the def keyword, followed by a function name, parentheses ( ), and a colon :. The function body is indented, and it may contain one or more statements. Functions can accept parameters and return values.

---

## Basic Structure of a Function

A function typically consists of:

- The def keyword to define the function.
- A function name (should be meaningful).
- Parentheses () (may contain parameters).
- A colon (:) to indicate the start of the function body.
- Indented block with function logic.
- An optional return statement to return a result.



## Keyword Arguments in function(kwargs)

Keyword arguments allow passing values with parameter names, making the function call more readable.

### Example: Keyword Arguments

```python
def describe_pet(animal="dog", name="Buddy"):
    print(f"I have a {animal} named {name}.")

describe_pet(animal="cat", name="Whiskers")
describe_pet(name="Max", animal="rabbit")  # Order doesn't matter
```

**Output:**

```css
I have a cat named Whiskers.
I have a rabbit named Max.
```

## Arbitrary Arguments (*args and **kwargs)

Python functions can accept a variable number of arguments.

### Example: *args for Multiple Positional Arguments

```python
def sum_all(*numbers):
    return sum(numbers)

print(sum_all(1, 2, 3, 4, 5))  # Output: 15
```

Here, `*numbers` collects all positional arguments into a tuple.

## Example: **kwargs for Multiple Keyword Arguments

```python
def display_info(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

display_info(name="Alice", age=25, city="New York")
```

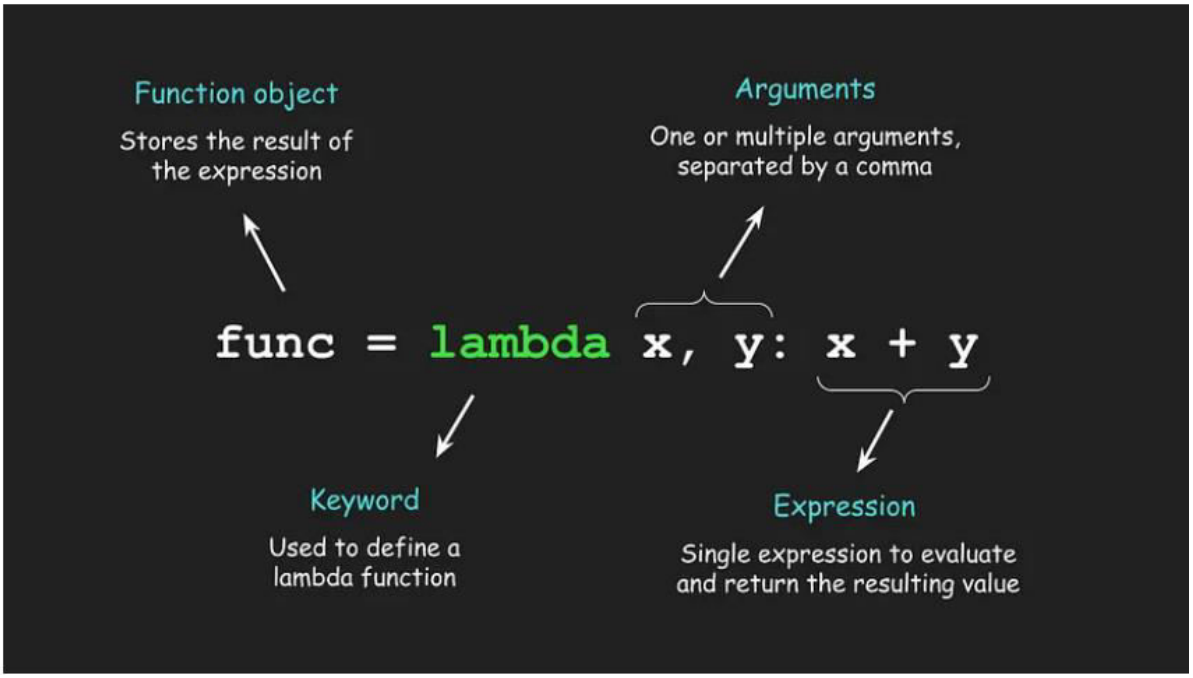
**Output:**

```vbnet
name: Alice
age: 25
city: New York
```

Here, `**info` collects all keyword arguments into a dictionary.

## Lambda Functions (Anonymous Functions)

Python provides lambda functions, which are small, anonymous functions defined using the lambda keyword.




## Example 1: Lambda vs Normal Function

### Using `def` Function

```python
def square(x):
    return x * x

print(square(5))  # Output: 25
```

### Using Lambda Function

```python
square = lambda x: x * x
print(square(5))  # Output: 25
```

✔ The lambda function is shorter and doesn't need a `return` statement.

## Why Use Lambda Functions in Python?

A lambda function is a small, anonymous function that is useful when you need a simple function for a short period, without defining it using def.

**Use Cases**:

- When you need a quick, one-time function.
- When a function has only one expression (a single return value).
- When passing a function as an argument to another function (e.g., in sorting, filtering, or mapping operations).

---

## Basic Syntax of Lambda Function

```python
lambda arguments: expression
```

- **lambda** keyword defines the function.
- **arguments** are the input values (like parameters in normal functions).
- **expression** is the computation that will be returned.

## Example 2: Lambda for Multiple Arguments

```python
add = lambda a, b: a + b
print(add(3, 7))  # Output: 10
```

## Example 3: Using Lambda in map(), filter(), sorted()

### Using map() to Square a List

```python
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x * x, numbers))
print(squared)  # Output: [1, 4, 9, 16, 25]
```

### Using filter() to Get Even Numbers

```python
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # Output: [2, 4, 6]
```

### Using sorted() with Lambda for Custom Sorting

```python
students = [("Alice", 25), ("Bob", 22), ("Charlie", 23)]
sorted_students = sorted(students, key=lambda x: x[1])  # Sort by age
print(sorted_students)
# Output: [('Bob', 22), ('Charlie', 23), ('Alice', 25)]
```

## Recursive Functions

A **recursive function** is a function that **calls itself** within its definition. It continues calling itself until a **base condition** (termination condition) is met.

### Why Use Recursion?

1. **Solves Problems That Have a Repetitive Pattern**
   - Recursion is useful for problems that can be broken down into **smaller subproblems** of the same type (e.g., factorial, Fibonacci, tree traversal).
2. **Makes Code More Readable**
   - Instead of using loops and extra variables, recursion provides a **cleaner and more logical approach**.
3. **Essential for Data Structures**
   - Recursion is widely used in **tree traversal**, **graph traversal**, and **divide & conquer algorithms** (e.g., QuickSort, MergeSort).

## Simple Example: Factorial Using Recursion

Factorial of a number ( `n!` ) is calculated as:

$$n! = n \times (n-1)!$$

With base condition: `1! = 1`

## Example: Recursive Function (Factorial Calculation)

```python
def factorial(n):
    if n == 1:  # Base case (stopping condition)
        return 1
    return n * factorial(n - 1)  # Recursive call

print(factorial(5))  # Output: 120
```

## How it works:

```matlab
factorial(5) = 5 * factorial(4)
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1 (Base case)
```

Final result:

`5 * 4 * 3 * 2 * 1 = 120`

## Local and Global Variables in Python

In Python, variables have different scopes, meaning their accessibility depends on where they are defined. The two main types of variables based on scope are:

1. **Local Variable** – Defined inside a function and accessible only within that function.
2. **Global Variable** – Defined outside all functions and accessible throughout the program.

## When to Use Local and Global Variables?

### Use Local Variables When:

- You need a variable only inside a function.
- You want to avoid accidental changes to global variables.
- You need memory-efficient code (local variables are deleted after function execution).

## Use Global Variables When:

- A value needs to be shared across multiple functions.
- You have configuration settings that are used throughout the program.
- You want to track application-wide data (e.g., user count, game score).

## Difference Between Local and Global Variables in Python

| Feature | Local Variable | Global Variable |
|---|---|---|
| **Scope** | Inside a function only | Throughout the entire program |
| **Defined Inside** | Function body ( `def` ) | Outside all functions |
| **Accessed By** | Only inside the function | Any function in the program |
| **Modification** | Cannot modify a global variable directly (unless `global` keyword is used) | Can be modified anywhere |
| **Lifetime** | Exists only while the function executes | Exists as long as the program runs |

## Example of Local and Global Variables in Python

```python
# Global Variable
x = 10

def my_function():
    # Local Variable
    y = 5
    print("Inside function, x (Global):", x)
    print("Inside function, y (Local):", y)

my_function()

# Trying to access y outside the function will cause an error
# print(y)  # This will give a NameError because y is local
```

**Output:**

```lua
Inside function, x (Global): 10
Inside function, y (Local): 5
```

## Modifying Global Variables Inside a Function

By default, if you try to modify a global variable inside a function, Python will treat it as a local variable and throw an error. To modify a global variable inside a function, use the global keyword.

## Example: Using global Keyword

```python
x = 10

def modify_global():
    global x  # Declare x as global
    x = 20
    print("Inside function, modified x:", x)

modify_global()
print("Outside function, x:", x)  # Value of x is changed globally
```

**Output:**

```bash
Inside function, modified x: 20
Outside function, x: 20
```

## Function Implementation in Real-Time

Functions are used in real-world applications to:

- **Calculate prices or totals**: A function that calculates tax, discount, and final price in an e-commerce platform.
- **Handle user input**: A function that validates user input in forms (e.g., checking email format).
- **Automating repetitive tasks**: Functions can automate tasks like data processing or logging.

**Function Architecture**

The architecture of a function in Python typically follows this structure:

1. **Function Definition**: Using def to define the function.
2. **Parameters (Inputs)**: Optional inputs the function will accept.
3. **Code Block**: The actual logic the function performs.
4. **Return Statement**: (Optional) Outputs a result to the caller.
5. **Function Call**: The function is executed when it's called by its name.

Example:

```
def function_name(parameter1, parameter2):
    # Logic block
    return result
```

**Future Advancements of Functions**

As Python evolves, future advancements could include:

- **Support for asynchronous functions**: With async/await, functions may perform tasks without blocking execution.

- **Type hinting improvements**: Functions could see more precise typing, enhancing code clarity and debugging.
- **Integration with machine learning**: Functions could become more integrated with data science and AI libraries like TensorFlow or PyTorch, allowing functions to be part of models

**Interview Questions and Answers About Functions**

BASIC LEVEL QUESTIONS

**Q1: What is the purpose of functions in Python?**

**Ans:** Functions in Python allow us to organize code into reusable blocks, making the code modular, easier to debug, and maintainable.

**Q2: Can you explain the difference between a function that returns a value and one that does not?**

**Ans:** A function that returns a value uses the return keyword to send a result back to the caller. A function that does not return anything ends with the return keyword without any value or simply returns None.

**Q3: What are the advantages of using functions in Python?**

**Ans:** Functions provide several advantages, such as:

- Code Reusability – Write once, use multiple times.
- Better Organization – Code is easier to read and maintain.
- Modularity – Breaks complex programs into smaller, manageable parts.
- Avoids Redundancy – Eliminates repeated code.
- Improves Debugging – Makes error-finding easier.

**Q4: What are the two types of functions in Python?**

**Ans:** Two types of function:

Built-in Functions – Functions provided by Python, e.g., print(), len(), sum().

User-defined Functions – Functions created by users using the def keyword.

**Q5: What is the difference between a function call and a function definition?**

| Term | Definition | Example |
|---|---|---|
| **Function Definition** | Creating a function using `def`. | `def greet(): print("Hello")` |
| **Function Call** | Executing a function using its name. | `greet()` |

**INTERMEDIATE LEVEL QUESTIONS**

**Q6: What is a lambda function?**

**Ans:** A lambda function is an anonymous function defined using the lambda keyword. It's typically used for short operations.

Example of lambda:

add = lambda x, y: x + y

**Q7: What is recursion in Python?**

**Ans:** Recursion is when a function calls itself in order to solve a problem. The function should have a base case to stop the recursion.

Example:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
```

## Q.8: What is the difference between map(), filter(), and reduce()?

| Function | Purpose | Example |
|---|---|---|
| **map()** | Applies a function to each item in a list. | `map(lambda x: x*2, [1,2,3])` |
| **filter()** | Filters elements based on a condition. | `filter(lambda x: x>2, [1,2,3])` |
| **reduce()** | Applies a function cumulatively. | `reduce(lambda x, y: x*y, [1,2,3])` |

## Q.9: What is the difference between built-in functions and user-defined functions?

| Category | Built-in Functions | User-defined Functions |
|---|---|---|
| **Definition** | Predefined in Python | Defined using `def` |
| **Examples** | `print()`, `len()`, `max()` | Custom functions |
| **Usage** | Used directly | Must be defined first |

## Q.10:Why are lambda functions used in Python?

**Ans**: Lambda functions are used:

- When a function is needed only once.
- Inside functions like map(), filter(), sorted().

## ADVANCED LEVEL QUESTIONS

### Q.11 What are function arguments in Python?

**Ans:**Function arguments are values passed to a function when calling it. Python supports:

1. **Positional Arguments** – Passed in order.
2. **Keyword Arguments** – Passed using parameter names.
3. **Default Arguments** – Have predefined values.
4. **Arbitrary Arguments** – *args (multiple positional args) and **kwargs (multiple keyword args).

### Q.12What is the difference between actual and formal arguments?

**Ans:**

| Type | Definition | Example |
|---|---|---|
| Actual Argument | Value passed while calling a function. | `add(5, 3)` |
| Formal Argument | Variable used inside function definition. | `def add(a, b): return a+b` |

### Q.13What are *args and **kwargs in Python?

**Ans:**In Python, *args and **kwargs are used in function definitions to handle variable-length arguments. This means they allow functions to accept any number of arguments without explicitly defining them.

- ***args**→ Allows multiple positional arguments.allows a function to accept **any number of positional arguments**. These arguments are **passed as a tuple** inside the function.

- ****kwargs→** Allows multiple keyword arguments.allows a function to accept **any number of keyword arguments**. These arguments are **passed as a dictionary** inside the function.

**Q.14 What is function overloading? Does Python support it?**

Function overloading means having multiple functions with the same name but different arguments. Python does not support function overloading directly, but we can achieve it using default arguments or *args.

```
def example(*args, **kwargs):
print(args)
print(kwargs)
example(1, 2, 3, name="Alice", age=25)
```

**Q.15 What is the base condition in recursion?**

**Ans:**A base condition is **the stopping condition** in recursion that prevents infinite function calls.

```
def countdown(n):
    if n == 0:  # Base condition
        return
    print(n)
countdown(n - 1)
```